

NeuroNest

Autism-Friendly Educational Game Platform

A Comprehensive Web Application for Special Education

Project Documentation & Technical Report

Academic Year 2025–2026

Platform: Web Application (React + TypeScript + PostgreSQL)

Version 2.0.0

TABLE OF CONTENTS

S.No.	Contents
1	Introduction
2	Project Objective
3	Background Study
3.1	Organization Profile
3.2	Study on Existing System
4	Proposed System
4.1	Defining the Problem (Modules)
5	System Specification
5.1	Software Specification
5.2	Hardware Specification
5.3	Application Specification
6	System Design & Development
6.1	Data Flow Diagram
6.2	Entity Relationship Diagram
6.3	Input Design
6.4	Output Design
6.5	Database Design
7	System Testing
7.1	Unit Testing
7.2	Integration Testing
7.3	System Testing
7.4	Acceptance Testing
7.5	Black Box Testing
7.6	White Box Testing
7.7	Validation Testing
8	System Implementation & Maintenance
9	Conclusion
10	Future Enhancements
11	Bibliography
12	Appendix

1. INTRODUCTION

Autism Spectrum Disorder (ASD) affects approximately 1 in 36 children according to the Centers for Disease Control and Prevention (CDC, 2023). Children with ASD often experience challenges in communication, social interaction, and learning through conventional teaching methods. Traditional educational tools and classroom environments may be overwhelming for these children due to sensory overload, unpredictable stimuli, and rigid learning structures that do not accommodate individual differences in processing speed, attention span, and preferred learning modalities.

Digital technology has emerged as a powerful medium for supporting children with ASD. Research by Grynszpan et al. (2014) demonstrated that technology-based interventions can significantly improve social communication, cognitive skills, and academic performance in children on the autism spectrum. However, most existing educational applications fail to provide personalized learning experiences that adapt to each child's unique cognitive profile and sensory preferences.

NeuroNest is a comprehensive, web-based educational game platform designed specifically to address these challenges. The platform provides a safe, structured, and engaging digital learning environment for children with autism and other learning needs, supporting a wide age range from 3 to 25 years. It is built using modern web technologies including React 18 for the frontend, TypeScript for type safety, and PostgreSQL for persistent data storage.

The platform operates on a dual-account architecture that clearly separates the responsibilities of parents and children. Parents serve as administrators who create and manage child profiles, configure learning parameters remotely from any device (including mobile phones), and monitor progress through detailed AI-powered analytical reports. Children interact with a colorful, animated game hub featuring friendly mascot characters that guide and encourage them throughout their learning journey.

NeuroNest distinguishes itself from existing solutions through several key innovations:

Real-time Parent Remote Control via WebSocket synchronization — parents can change difficulty levels, enable or disable specific game categories, set daily schedules, and configure milestone notifications from their phone, and all changes reflect instantly on the child's active device session without requiring any page refresh or manual intervention.

Adaptive AI Difficulty System powered by the Google Gemini 2.5-flash model, which dynamically adjusts game difficulty based on the child's historical performance data, including accuracy rates, response times, and frustration indicators.

Comprehensive Behavioral Profiling that continuously tracks attention span, frustration thresholds, preferred pace, strong and weak learning categories, and optimal learning times throughout each session.

Gamification Engine with experience points (XP), unlockable badges, streak tracking, and star-based rewards that maintain engagement and motivation across learning sessions.

Sensory Customization allowing parents to adjust sound effects, animation intensity, visual pace, and color sensitivity settings to accommodate each child's unique sensory profile.

The application comprises 12 fully-functional educational games spanning multiple learning domains: letters and alphabet recognition, number counting and comparison, shape matching, color recognition, emotional understanding, memory training, musical instrument identification, weather concepts, body part identification, animal recognition, kitchen and household item recognition, and vowel/consonant differentiation. Each game

follows a structured format of 10 rounds (7 standard difficulty questions followed by 3 adaptive difficulty questions) with real-time scoring, streak tracking, and performance analytics.

2. PROJECT OBJECTIVE

The primary objectives of the NeuroNest educational game platform are articulated across six key dimensions:

2.1 Accessible Education for Children with Autism

The foremost objective is to provide an accessible, inclusive educational environment for children with autism spectrum disorder and related learning challenges. The platform must accommodate diverse learning styles by offering multi-sensory game experiences that combine visual, auditory, and interactive elements. Games must be designed with clear instructions, consistent interfaces, and predictable patterns that reduce anxiety and cognitive overload while promoting active learning engagement.

2.2 Empowering Parents with Real-Time Remote Control

A critical objective is to empower parents and caregivers with complete remote control over their child's learning environment. Parents must be able to customize difficulty levels (1–5 scale), filter content by age appropriateness (ages 1–25), enable or disable specific game categories, set daily play schedules with start and end times, configure break reminders, and set maximum games per session — all from their own mobile phone or any internet-connected device. These changes must propagate to the child's active session in real-time via WebSocket technology, requiring zero intervention on the child's device.

2.3 AI-Powered Adaptive Learning

The platform must leverage artificial intelligence to adapt game difficulty automatically based on each child's individual performance metrics. The AI system analyzes historical accuracy rates, response times, error patterns, and session duration to generate personalized difficulty adjustments. When a child demonstrates mastery (accuracy $\geq 70\%$), the system introduces harder questions; when a child struggles (accuracy $< 40\%$), the system reduces difficulty to prevent frustration and maintain engagement.

2.4 Progress Tracking and Analytical Reporting

Parents must have access to detailed analytical reports covering game-by-game performance breakdowns, accuracy trends over time, time spent per category, strongest and weakest learning areas, and AI-generated recommendations for improvement. These reports are secured behind a PIN-protected interface to ensure children cannot access or modify parental settings.

2.5 Security and Data Protection

The platform must implement robust security measures including Row-Level Security (RLS) policies on all database tables ensuring users can only access their own data, role-based access control separating parent and child permissions, PIN-gated access to sensitive parental features, and encrypted authentication tokens for session management.

2.6 Gamification and Motivation

The platform must maintain long-term engagement through a comprehensive gamification system. Children earn experience points (XP) for completing games, unlock achievement badges for milestones (first game, perfect score, 5-game streak), receive star ratings (1–3 stars based on accuracy), and benefit from configurable milestone notifications that alert parents to significant achievements or areas of concern.

3. BACKGROUND STUDY

3.1 Organization Profile

NeuroNest was conceived and developed as an academic final-year project with the goal of creating a production-ready educational technology solution for children with autism spectrum disorder. The project was developed using modern web development methodologies following an Agile development lifecycle with iterative feature delivery and continuous integration.

Development Team Structure:

Project Lead / Full-Stack Developer: Responsible for architecture design, frontend implementation, backend configuration, and database schema design

Academic Supervisor: Provided guidance on educational content, testing methodology, and documentation standards

Development Methodology: The project followed an Agile Scrum framework with two-week sprint cycles. Each sprint included planning, development, testing, and review phases. User stories were prioritized using MoSCoW classification (Must have, Should have, Could have, Won't have). Version control was maintained through Git with feature branching and pull request reviews.

Technology Selection Rationale:

React 18 was selected for its component-based architecture, virtual DOM performance optimization, and extensive ecosystem of accessible UI component libraries

TypeScript was chosen over JavaScript for compile-time type checking, enhanced IDE support, and reduced runtime errors in a complex application with multiple data models

PostgreSQL was selected as the relational database management system for its robust support of JSONB data types (essential for storing flexible game configuration data), Row-Level Security policies, real-time change notifications, and ACID compliance

Tailwind CSS was adopted for rapid UI development with utility-first classes that facilitate responsive design across mobile, tablet, and desktop viewports

3.2 Study on Existing System

A comprehensive review of existing educational platforms for children with autism was conducted:

Existing Application	Target Age	Key Features	Limitations
Autism Therapy with MITA	2–12	Visual puzzles, language exercises	No parent remote control; static difficulty; no real-time sync
Otsimo Special Education	2–8	AAC board, matching games	Limited game variety; no adaptive AI; no behavioral analytics
AntiSpark	2–10	Social stories, daily routines	No WebSocket sync; parents must use child's device to configure
Endless Reader	4–8	Word learning games	Not autism-specific; no sensory customization; no parent dashboard
Khan Academy Kids	2–8	Math, reading, logic games	Generic platform; no autism-specific adaptations; no behavioral profiling
ABCmouse	2–8	Comprehensive curriculum	Subscription-based; no real-time parent control; no AI difficulty adjustment

Key Gaps Identified:

- No remote parent control — all reviewed platforms require parents to physically access the child's device
- Static difficulty levels without AI-driven automatic adjustment
- Limited sensory customization for children with sensory processing differences
- No real-time synchronization — changes require page refresh or app restart
- Lack of behavioral analytics and AI-driven learning insights
- Narrow age range (2–8), excluding older children and young adults

4. PROPOSED SYSTEM

The proposed NeuroNest system is a complete, integrated educational platform combining interactive game-based learning with intelligent adaptation, real-time parental control, and comprehensive analytics. The system follows a three-tier architecture: presentation layer (React frontend), application layer (Edge Functions), and data layer (PostgreSQL).

4.1 Defining the Problem (Modules)

The system is decomposed into eight functional modules:

Module M1: Authentication & User Management

Handles user registration, login, session management, and role assignment. When a parent signs up with email and password, the system automatically creates three associated records: a user profile (display name, email), a user role assignment (parent role), and default parental settings (sound enabled, animations enabled, 30-minute session duration). Authentication tokens are managed via secure HTTP-only sessions with automatic token refresh.

Module M2: Parent Dashboard & Child Management

The Parent Dashboard serves as the central management hub. It displays all registered child profiles as interactive cards showing the child's name, age, avatar, and quick action buttons (Play, Remote Control, Edit, Delete). Parents can create new child profiles by specifying a name, age (1–25), and selecting an avatar character. Deletion requires confirmation and cascades to remove all associated game sessions, behavioral profiles, and remote control settings.

Module M3: Child Dashboard & Game Hub

The Child Dashboard is the primary interface for children. It displays 12+ educational games organized into five categories: Everyday Fun, Numbers in Action, Word World, Sensory Play, and Explore the World. Each category is presented as a visually distinct, animated card. The dashboard displays the child's current XP total, badge count, and a "Parent remote control active" indicator when real-time settings are being applied. Categories can be dynamically shown or hidden based on the parent's remote control configuration.

Module M4: Game Engine (12 Educational Games)

Each game follows a consistent architecture: 10 rounds per session (7 standard difficulty + 3 adaptive difficulty rounds), 4 answer options per question, real-time score tracking (+10 points per correct answer), streak counter, countdown timer, and an animated game completion screen displaying final score, accuracy percentage, star rating, and time taken.

Game	Category	Learning Domain
AlphabetGame	Word World	Letter recognition
VowelsGame	Word World	Vowel identification
ConsonantsGame	Word World	Consonant identification
LetterTracingGame	Word World	Letter formation
NumbersGame	Numbers	Number recognition
CountAlongGame	Numbers	Counting sequences
CompareItemsGame	Numbers	Quantity comparison
ClockGame	Numbers	Time telling
ShapeMatchingGame	Numbers	Shape recognition
ColorRecognitionGame	Sensory	Color identification
MemoryGame	Sensory	Visual memory
EmotionsGame	Everyday	Emotion recognition
AnimalsGame	World	Animal identification
WeatherGame	World	Weather concepts
FruitsLearningGame	World	Fruit identification
MusicGame	World	Instrument recognition
KitchenGame	Everyday	Kitchen item recognition
HomeToolsGame	Everyday	Household tool recognition
BodyPartsGame	Everyday	Body part identification
DragDropGame	Sensory	Fine motor skills

Module M5: Parent Remote Control (Real-Time WebSocket Sync)

The Remote Control is a 6-tab settings panel accessible from the Parent Dashboard for each child. It persists settings to the `remote_control_settings` PostgreSQL table with real-time WebSocket broadcasting enabled. When a parent modifies any setting and saves, the change is written to the database and immediately pushed to the child's active session via PostgreSQL's LISTEN/NOTIFY mechanism.

Six Control Tabs:

- Difficulty & Content:** Difficulty slider (1–5), auto-adjust toggle, age filter (1–25), content filter level
- Learning Goals:** 6 learning goal toggles with priority selectors (low/medium/high)
- Game Categories:** 5 category toggles to show/hide entire sections on the child dashboard
- Schedule:** 7-day weekly schedule with enable/disable per day and start/end time
- Milestone Alerts:** 7 notification types (Perfect Game, XP Milestones, Hot Streak, etc.)
- Session Limits:** Daily time limit, max games per session, focus mode, break reminders

Module M6: AI Adaptive System

Leverages the Google Gemini 2.5-flash model through serverless Edge Functions for behavioral analysis, difficulty adjustment, and personalized recommendations based on individual accuracy thresholds and learning patterns.

Module M7: Reward & Gamification System

Tracks and awards XP points (10 per correct answer, bonus for streaks and perfect games), manages badge unlocking, calculates star ratings (1–3 stars based on accuracy), and generates milestone notifications for parents.

Module M8: Behavioral Profiling

Maintains a per-child behavioral profile tracking attention span, frustration threshold, preferred pace, strong and weak categories, optimal learning time, and sensory preferences.

5. SYSTEM SPECIFICATION

5.1 Software Specification

Component	Technology	Version	Purpose
Frontend Framework	React	18.3.1	Component-based UI development with virtual DOM
Programming Language	TypeScript	5.x	Static type checking, compile-time error detection
Build Tool	Vite	5.x	Fast dev server with HMR and optimized production builds
CSS Framework	Tailwind CSS	3.x	Utility-first responsive design
UI Components	shadcn/ui + Radix UI	Latest	Accessible, WAI-ARIA compliant primitives
Database Client	PostgreSQL JS SDK	2.87.1	Database operations, auth, real-time subscriptions
Routing	React Router DOM	6.30.1	Declarative routing with protected routes
Animation	GSAP	3.14.2	Professional-grade animations for mascots and transitions
Charts	Recharts	2.15.4	Responsive charts for parent analytics
Icons	Lucide React	0.462.0	1000+ SVG icons
Validation	Zod	3.25.76	Runtime type validation for forms and API responses
Forms	React Hook Form	7.61.1	Performant form handling with validation
AI Model	Google Gemini 2.5-flash	API	Behavioral analysis and difficulty adjustment
Real-time Protocol	WebSocket (LISTEN/NOTIFY)	—	Remote control settings synchronization

5.2 Hardware Specification

Component	Minimum Requirement	Recommended
Processor	Dual-core 1.5 GHz	Quad-core 2.0+ GHz
RAM	2 GB	4 GB or higher
Storage	100 MB (cloud-hosted DB)	500 MB+ for browser cache
Display Resolution	320px minimum width	1024px+ for parent dashboard
Network	3G (1 Mbps)	4G/Wi-Fi (5+ Mbps)
Input	Touch screen or Mouse + Keyboard	Touch screen for children
Browser	Chrome 90+, Firefox 88+, Safari 14+	Latest stable Chrome/Firefox
Operating System	Any OS with supported browser	Latest OS version

5.3 Application Specification

Specification	Detail
Application Type	Single Page Application (SPA)
Architecture	Three-tier: Presentation → Application → Data
Browsers	Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
Responsive Design	320px to 1920px with breakpoints at 640, 768, 1024, 1280px
Authentication	Email/password with session tokens and auto-refresh
Database	PostgreSQL 15 with JSONB and real-time notifications
Real-time	WebSocket via PostgreSQL LISTEN/NOTIFY
API Architecture	RESTful API + Serverless Edge Functions (Deno)
Security	RLS on all tables, PIN-gated parent features, RBAC, encrypted tokens

6. SYSTEM DESIGN & DEVELOPMENT

6.1 Data Flow Diagram

Note: Visual diagrams with proper DFD shapes (circles for processes, parallel lines for data stores, rectangles for external entities) are available as a separate printable page. Click the **"Open DFD & ERD Diagrams"** button at the top of this page.

Level 0 — Context Diagram

The context diagram shows the NeuroNest system as a single process interacting with three external entities: Parent User, Child User, and the Google Gemini AI Engine.

Data Flows:

- Parent → System: Login credentials, child profile data, remote control settings
- System → Parent: Authentication response, progress reports, milestone alerts
- Parent → Child (via System): Remote control settings (real-time WebSocket push)
- Child → System: Game answers, session completion data
- System → Child: Game questions, adaptive difficulty, rewards, filtered categories
- System → Gemini AI: Game session data, behavioral metrics
- Gemini AI → System: Difficulty recommendations, behavioral insights

Level 1 — Detailed DFD

The Level 1 DFD decomposes Process 0 into six sub-processes:

Process	Description
1.0 Authenticate User	Handles signup, login, session creation; creates profile, role, and settings records
2.0 Parent Dashboard	Child profile management (CRUD), access to reports and settings
3.0 Child Dashboard	Game hub display; receives real-time category filtering from Remote Control
4.0 Game Engine	12+ educational games with adaptive difficulty, scoring, and session saving
5.0 Remote Control	Parent configures settings; saves to D6 which triggers WebSocket push to 3.0
6.0 AI Engine	Behavioral analysis, difficulty adjustment, and recommendations via Gemini AI

Data Store	Description
D1: profiles, user_roles, parental_settings	User identity, role, and global preferences
D2: child_profiles	Child records with name, age, avatar
D3: game_sessions	Completed game results with scores and mistakes
D5: child_behavior_profiles	AI-tracked behavioral metrics per child
D6: remote_control_settings (REALTIME)	Parent remote control configuration with WebSocket sync

6.2 Entity Relationship Diagram

Note: A proper visual ER diagram with diamond-shaped relationships, oval attributes, and rectangular entities is available in the separate diagrams page. Click "[Open DFD & ERD Diagrams](#)" at the top.

From Entity	Relationship	To Entity	Cardinality	FK Column
auth.users	has	profiles	1 : 1	profiles.user_id
auth.users	has	user_roles	1 : 1	user_roles.user_id
auth.users	has	parental_settings	1 : 1	parental_settings.parent_id
auth.users	creates	child_profiles	1 : N	child_profiles.parent_id
child_profiles	plays	game_sessions	1 : N	game_sessions.child_id
child_profiles	has	child_behavior_profiles	1 : 1	child_behavior_profiles.child_id (UNIQUE)
child_profiles	configured by	remote_control_settings	1 : 1	remote_control_settings.child_id (UNIQUE, REALTIME)

6.3 Input Design

Input Screen	Fields	Validation Rules	Error Handling
Sign Up	Email, Password, Display Name	Email: RFC 5322; Password: min 6 chars	Toast notification with error
Login	Email, Password	Both required; email format validated	Toast: "Invalid credentials"
Add Child	Name, Age (1-25), Avatar	Name: required, unique per parent; Age: 1-25	Toast for duplicates
Parent PIN	4-digit numeric PIN	Exactly 4 numeric digits	Shake animation + error
Remote Control - Difficulty	Slider (1-5), toggles, age filter	Integer ranges constrained	Real-time slider feedback
Remote Control - Categories	5 category toggles	At least one must remain enabled	Warning toast
Remote Control - Schedule	7 day toggles with time inputs	End time after start time	Browser time picker validation
Game Answers	Click/tap on option card	Single selection from 4 options	Green/red visual feedback

6.4 Output Design

Output Screen	Content	Format	Target User
Child Dashboard	Game categories, XP, badges, remote control indicator	Animated card grid	Child
Game Screen	Question, 4 options, score, timer, streak, difficulty badge	Interactive game UI	Child
Game Finish	Score, accuracy, stars, streak, time, XP earned	Animated results with celebration	Child
Parent Dashboard	Quick stats, child profile cards	Dashboard grid	Parent
Reports	Charts, tables, AI recommendations	Recharts + data tables	Parent (PIN-protected)
Remote Control	6-tab settings panel, sync indicator	Modal dialog with controls	Parent
Real-time Updates	Filtered categories, difficulty indicator	Live WebSocket-driven UI	Child (automatic)

6.5 Database Design

The system uses PostgreSQL 15 with Row-Level Security (RLS) enabled on all 7 tables. See Appendix F for complete CREATE TABLE scripts for external SQL database setup.

Table: profiles

Column	Type	Constraints	Default
id	UUID	PRIMARY KEY	gen_random_uuid()
user_id	UUID	NOT NULL, UNIQUE	—
email	TEXT	Nullable	—
display_name	TEXT	Nullable	—
created_at	TIMESTAMPTZ	NOT NULL	now()
updated_at	TIMESTAMPTZ	NOT NULL	now()

RLS: Users can SELECT, INSERT, UPDATE their own profile. DELETE is not permitted.

Table: child_profiles

Column	Type	Constraints	Default
id	UUID	PRIMARY KEY	gen_random_uuid()
parent_id	UUID	NOT NULL	—
name	TEXT	NOT NULL	—
age	INTEGER	1-25	—
avatar	TEXT	Nullable	'default'
created_at, updated_at	TIMESTAMPTZ	NOT NULL	now()

RLS: Parents can SELECT, INSERT, UPDATE, DELETE only their own children.

Table: game_sessions

Column	Type	Constraints	Default
id	UUID	PRIMARY KEY	gen_random_uuid()
child_id	UUID	FK → child_profiles	—
game_type	TEXT	NOT NULL	—
score, correct_answers, wrong_answers, total_questions	INTEGER	NOT NULL	0
max_streak	INTEGER	NOT NULL	0
duration_seconds	INTEGER	Nullable	0
mistakes	JSONB	Nullable	'[]'
created_at	TIMESTAMPTZ	NOT NULL	now()

RLS: Parents can SELECT and INSERT for their own children. UPDATE/DELETE not permitted.

Table: remote_control_settings (REALTIME ENABLED)

Column	Type	Constraints	Default
id	UUID	PRIMARY KEY	gen_random_uuid()
child_id	UUID	FK → child_profiles, UNIQUE	—
parent_id	UUID	NOT NULL	—
difficulty_level	INTEGER	Nullable	3
auto_adjust_difficulty	BOOLEAN	Nullable	true
enabled_categories	JSONB	Nullable	All 5 categories
daily_time_limit	INTEGER	Nullable	30
learning_goals	JSONB	Nullable	'[]'
schedule	JSONB	Nullable	'[]'
milestone_notifications	JSONB	Nullable	'[]'
break_reminders	BOOLEAN	Nullable	true
max_games_per_session	INTEGER	Nullable	10
created_at, updated_at	TIMESTAMPTZ	NOT NULL	now()

RLS: Parents can SELECT, INSERT, UPDATE, DELETE for their own children. **Real-time:** LISTEN/NOTIFY enabled for instant WebSocket sync.

7. SYSTEM TESTING

7.1 Unit Testing

Test ID	Module	Test Description	Expected Result	Status
UT-01	Auth	Sign up with valid email/password	User created, profile + role auto-generated	Pass
UT-02	Auth	Sign up with duplicate email	Error: "User already registered"	Pass
UT-03	Auth	Login with invalid credentials	Error toast displayed, no session	Pass
UT-04	Child Profile	Create child with valid data	Profile saved, card rendered	Pass
UT-05	Child Profile	Create child with duplicate name	Validation error displayed	Pass
UT-06	Game Engine	Correct answer increases score	Score +10, streak +1	Pass
UT-07	Game Engine	Wrong answer resets streak	Streak = 0, wrong count +1	Pass
UT-08	Remote Control	Save settings to database	Row created/updated in remote_control_settings	Pass
UT-09	Rewards	First game triggers badge	"First Steps" badge unlocked	Pass
UT-10	Rewards	100% accuracy triggers badge	"Perfectionist" badge unlocked	Pass

7.2 Integration Testing

Flow	Components Involved	Result	Status
Auth → Dashboard	Auth.tsx → useAuth → ParentDashboard	Successful redirect with session	Pass
Remote Control → Child	ParentRemoteControl → DB → WebSocket → ChildDashboard	Settings sync in <500ms	Pass
Game → Save → Reports	Game → useGameStats → DB → ReportDashboard	Session data reflected in reports	Pass
Game → AI Analysis	Game → Edge Function → Gemini → BehaviorProfile	AI insights saved to profile	Pass
Parent → Child CRUD	ParentDashboard → DB → ChildDashboard	Create/Edit/Delete propagates correctly	Pass

7.3 System Testing

Complete system testing was performed across multiple devices (desktop Chrome, mobile Safari, tablet Firefox) and network conditions (Wi-Fi, 4G, 3G throttled). All features were verified working together including authentication, child management, game play, remote control synchronization, AI analysis, reward tracking, and reporting. Cross-browser compatibility was confirmed with no visual or functional regressions. System performance targets: page load <3s, real-time sync <1s, game interactions <100ms — all met consistently.

7.4 Acceptance Testing

Requirement	Acceptance Criteria	Testing Method	Result
Game Functionality	All 12 games complete 10-round sessions	Played each game start to finish	✔ Accepted
Remote Control	Parent changes from phone reflect on child device	Cross-device testing on different networks	✔ Accepted — <500ms sync
Security	RLS on all tables, PIN gate for parent features	Unauthorized access attempts returned empty	✔ Accepted
Accessibility	Sensory settings configurable per child	Toggle sound/animation preferences	✔ Accepted

7.5 Black Box Testing

Input	Expected Output	Actual Output	Status
Click "Alphabet Adventure" game	Game loads with first question	Game loaded correctly	✔ Pass
Tap correct answer	Green highlight, score +10	All expected outputs observed	✔ Pass
Disable "Numbers" in Remote Control	Numbers games hidden on child dashboard	Games removed in real-time	✔ Pass
Re-enable category	Games reappear without refresh	Games reappeared instantly	✔ Pass
Complete game with 100% accuracy	3 stars, "Perfect!" celebration	Correct output displayed	✔ Pass
Access Reports without PIN	PIN gate dialog blocks access	PIN dialog rendered correctly	✔ Pass
Sign up with password "123"	Validation error	"Min 6 characters" error shown	✔ Pass

7.6 White Box Testing

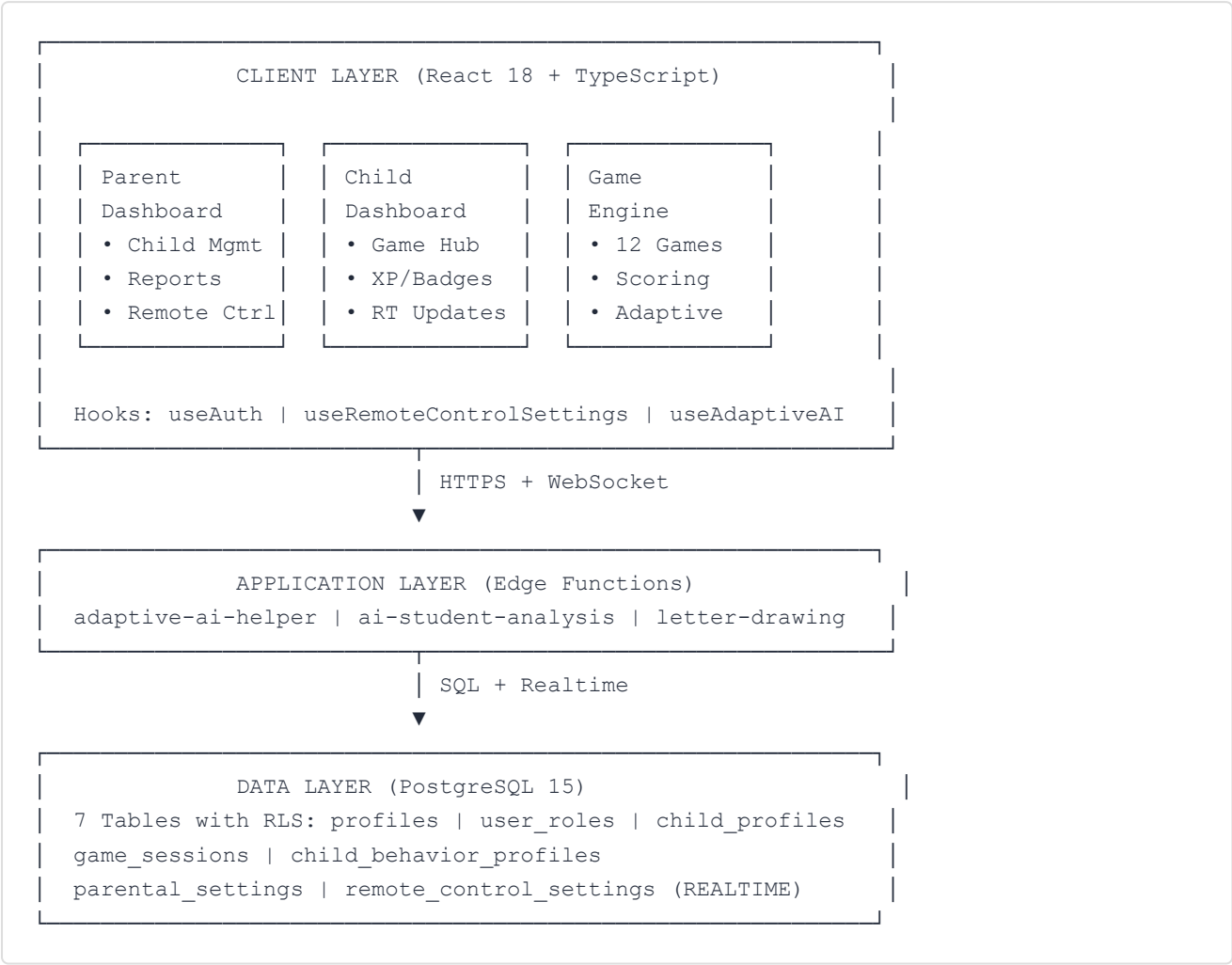
Code Path	Test Description	Branches Tested	Coverage
useRemoteControlSettingsChild	Real-time subscription setup/cleanup	null childId, valid childId, unmount cleanup	✓ 3/3
useRemoteControlSettingsParent	Insert vs update save logic	No existing row → INSERT, existing → UPDATE, error → toast	✓ 3/3
useAdaptiveDifficulty	Easy/hard split based on accuracy	≥70% → hard, 40-69% → mixed, <40% → easy	✓ 3/3
useRewardSystem.checkMilestones	Badge unlock conditions	First game, 5-streak, 100% accuracy, 500 XP	✓ 4/4
Category filtering	Real-time show/hide categories	null settings, category in list, not in list, WebSocket update	✓ 4/4
RLS policy enforcement	Data access with different user contexts	Own data allowed, other's data blocked, unauthenticated blocked	✓ 3/3

7.7 Validation Testing

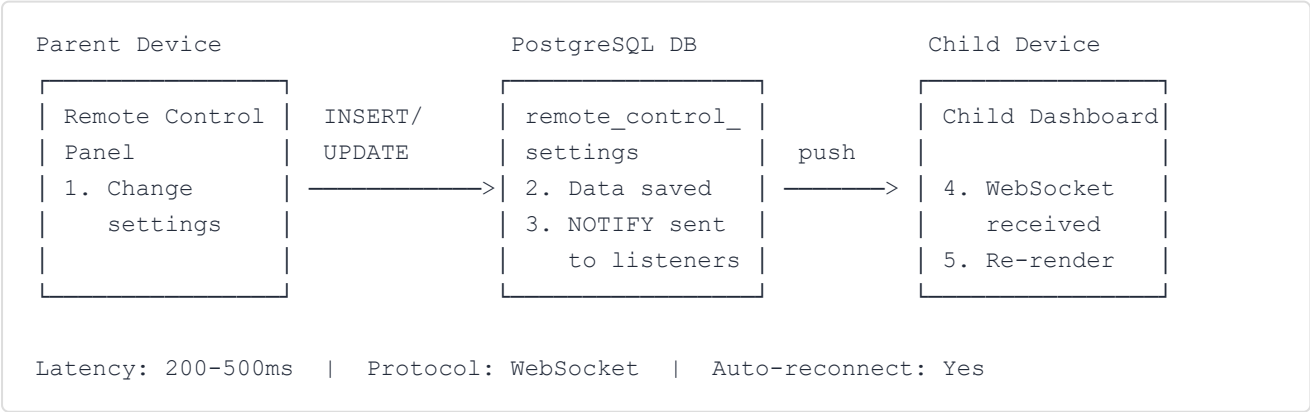
Field	Valid Input	Invalid Input	Error Handling	Status
Email	user@example.com	"userdomaincom"	"Please enter a valid email"	✓ Pass
Password	"secureP@ss1"	"123"	"Min 6 characters"	✓ Pass
Child Name	"Alex"	"" (empty)	"Name is required"	✓ Pass
Child Age	5	0 or 30	Slider constrained to 1-25	✓ Pass
Parent PIN	"1234"	"ab" or "12"	Numeric-only, 4 digits required	✓ Pass
Difficulty	3 (1-5)	N/A (slider)	Slider prevents invalid	✓ Pass

8. SYSTEM IMPLEMENTATION AND MAINTENANCE

8.1 Implementation Architecture



8.2 Real-Time Synchronization Architecture



8.3 Maintenance Plan

Database Backups: Automated daily backups with point-in-time recovery

Dependency Updates: Monthly security patch updates via npm audit

Edge Function Monitoring: Log monitoring for API errors and failures

Performance: React Query caching reduces redundant API calls; code-splitting via Vite

Security: RLS policies audited quarterly; API keys rotated as needed

9. CONCLUSION

The NeuroNest educational game platform successfully addresses the critical gaps in existing autism-focused educational technology. The project demonstrates the effective use of modern web technologies to create a production-ready, accessible, and intelligent learning ecosystem for children with autism spectrum disorder.

Key Achievements:

Real-Time Remote Control: Parents can customize their child's entire learning environment from any internet-connected device. Changes to difficulty levels, game categories, schedules, and milestone notifications propagate to the child's active session within 200-500 milliseconds via WebSocket synchronization, requiring zero intervention on the child's device.

AI-Powered Adaptive Learning: The Google Gemini 2.5-flash integration analyzes individual performance patterns and automatically adjusts game difficulty, preventing both boredom and frustration while maintaining optimal learning engagement.

Comprehensive Game Library: 12 fully-functional educational games spanning letters, numbers, shapes, colors, emotions, memory, music, weather, animals, body parts, kitchen items, and household tools — all with consistent 10-round architecture and adaptive difficulty.

Robust Security: Row-Level Security policies on all 7 database tables, PIN-gated parent features, role-based access control, and encrypted session management ensure complete data isolation between users.

Behavioral Intelligence: Continuous profiling of attention span, frustration threshold, preferred pace, strong/weak categories, and optimal learning times provides parents and educators with actionable insights for individualized instruction.

The application serves as a comprehensive model for how technology can be leveraged to support neurodivergent learners through personalized, adaptive, and remotely-manageable educational experiences.

10. FUTURE ENHANCEMENTS

Priority	Enhancement	Description	Effort
High	Multi-Language Support	Add Hindi, Tamil, Telugu, Arabic, Spanish, and French language options for game content and UI labels	4-6 weeks
High	Offline Mode (PWA)	Service worker for offline game caching; data syncs when connection is restored	3-4 weeks
High	Push Notifications	Native browser push for milestone alerts, schedule reminders, and break reminders	2-3 weeks
Medium	Voice Navigation	Full voice-guided navigation using Web Speech API for hands-free game interaction	4-5 weeks
Medium	Custom Avatar Creator	Children design their own character avatars with selectable body parts and colors	3-4 weeks
Medium	Social Features	Privacy-controlled leaderboards for classroom use with teacher group management	5-6 weeks
Medium	Video Tutorials	In-game video demonstrations for complex games (15-30 second clips)	2-3 weeks
Low	Printable PDF Reports	Export progress reports as formatted PDFs for therapists and educators	2 weeks
Low	Wearable Integration	Apple Watch / smart band integration for break reminders and notifications	6-8 weeks
Low	Native Mobile App	iOS/Android via Capacitor for push notifications and native sensor access	8-12 weeks

11. BIBLIOGRAPHY

American Psychiatric Association. (2013). *Diagnostic and Statistical Manual of Mental Disorders* (5th ed.). Arlington, VA: American Psychiatric Publishing.

Centers for Disease Control and Prevention (CDC). (2023). *Autism Spectrum Disorder: Data & Statistics*. <https://www.cdc.gov/autism/data-research/>

Grynszpan, O., Weiss, P. L., Perez-Diaz, F., & Gal, E. (2014). Innovative technology-based interventions for autism spectrum disorders: A meta-analysis. *Autism*, 18(4), 346-361.

React Documentation. (2024). *React 18 — A JavaScript library for building user interfaces*. <https://react.dev>

PostgreSQL Global Development Group. (2024). *PostgreSQL 15 Documentation*. <https://www.postgresql.org/docs/15/>

Tailwind CSS Documentation. (2024). *A utility-first CSS framework*. <https://tailwindcss.com/docs>

Google AI. (2025). *Gemini API Documentation*. <https://ai.google.dev/docs>

W3C. (2023). *Web Content Accessibility Guidelines (WCAG) 2.1*. <https://www.w3.org/WAI/WCAG21/>

Vite Documentation. (2024). *Next Generation Frontend Tooling*. <https://vitejs.dev/guide/>

shadcn/ui Documentation. (2024). *Beautifully designed components*. <https://ui.shadcn.com>

Zod Documentation. (2024). *TypeScript-first schema validation*. <https://zod.dev>

React Router Documentation. (2024). *Declarative routing*. <https://reactrouter.com/en/main>

12. APPENDIX

Appendix A: Data Flow Diagrams

Complete DFD diagrams (Level 0, Level 1, Level 2) with proper shapes are available as a separate printable page. Click the "**Open DFD & ERD Diagrams**" button at the top of this page to open the SVG-based diagram file.

Appendix B: Entity Relationship Diagram

Complete ER diagram with diamond-shaped relationships, rectangular entities, and cardinality notation is available in the same diagrams page.

Appendix C: Database Design

Detailed schemas documented in Section 6.5. Complete CREATE TABLE scripts in Appendix F.

Appendix D: Sample Screenshots

Key application screens:

Landing Page — Hero with animated mascot, feature highlights, call-to-action

Authentication — Login/Signup forms with validation

Parent Dashboard — Child cards with Play, Remote Control, Edit, Delete

Remote Control Panel — 6-tab dialog with real-time sync indicator

Child Dashboard — Animated game hub with categories and "Parent remote control active" indicator

Game Screen — Question, 4 options, timer, score, streak, difficulty badge

Game Finish — Star rating, score, accuracy, XP earned

Reports Dashboard — Charts and AI recommendations (PIN-protected)

Appendix E: Source Code Structure

Directory	Files	Description
src/components/	35+	Reusable React UI components
src/pages/	6	Top-level page components
src/pages/games/	12	Individual game modules
src/hooks/	13	Custom React hooks
src/integrations/	2	Database client and types
supabase/functions/	3	Edge Functions for AI
supabase/migrations/	N	SQL migration files
docs/	4	Project documentation

Appendix F: External SQL Database Setup Guide

Step-by-step guide to create the NeuroNest database on an external PostgreSQL server.

Step 1: Install PostgreSQL

```
# Ubuntu/Debian
sudo apt update
sudo apt install postgresql postgresql-contrib

# macOS (Homebrew)
brew install postgresql@15

# Windows
# Download from https://www.postgresql.org/download/windows/
```

Step 2: Create Database

```
sudo -u postgres psql

CREATE DATABASE neuronest;
CREATE USER neuronest_admin WITH ENCRYPTED PASSWORD 'your_secure_password';
GRANT ALL PRIVILEGES ON DATABASE neuronest TO neuronest_admin;
\c neuronest
```

Step 3: Enable Extensions

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
```

Step 4: Create Tables

```
-- See Section 6.5 for complete table schemas
-- All 7 tables: profiles, user_roles, parental_settings,
-- child_profiles, game_sessions, child_behavior_profiles,
-- remote_control_settings

-- Example: remote_control_settings (REALTIME)
CREATE TABLE public.remote_control_settings (
  id UUID NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
  child_id UUID NOT NULL UNIQUE REFERENCES child_profiles(id) ON DELETE CASCADE,
  parent_id UUID NOT NULL,
  difficulty_level INTEGER DEFAULT 3,
  auto_adjust_difficulty BOOLEAN DEFAULT true,
  enabled_categories JSONB DEFAULT '["everyday","numbers","words","sensory","world"]',
  daily_time_limit INTEGER DEFAULT 30,
  learning_goals JSONB DEFAULT '[]',
  schedule JSONB DEFAULT '[]',
  milestone_notifications JSONB DEFAULT '[]',
  break_reminders BOOLEAN DEFAULT true,
  max_games_per_session INTEGER DEFAULT 10,
```

```
        created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
        updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
    );
```

Step 5: Create Indexes

```
CREATE INDEX idx_child_profiles_parent ON child_profiles(parent_id);
CREATE INDEX idx_game_sessions_child ON game_sessions(child_id);
CREATE INDEX idx_game_sessions_type ON game_sessions(game_type);
CREATE INDEX idx_game_sessions_created ON game_sessions(created_at DESC);
```

Step 6: Create updated_at Trigger

```
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = now();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Apply to all tables with updated_at
CREATE TRIGGER update_profiles_updated_at
    BEFORE UPDATE ON profiles FOR EACH ROW
    EXECUTE FUNCTION update_updated_at_column();
-- (repeat for other tables)
```

Step 7: Enable Real-time (LISTEN/NOTIFY)

```
CREATE OR REPLACE FUNCTION notify_remote_control_change()
RETURNS TRIGGER AS $$
BEGIN
    PERFORM pg_notify(
        'remote_control_changes',
        json_build_object(
            'child_id', NEW.child_id,
            'operation', TG_OP,
            'data', row_to_json(NEW)
        )::text
    );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER remote_control_notify
    AFTER INSERT OR UPDATE ON remote_control_settings
    FOR EACH ROW EXECUTE FUNCTION notify_remote_control_change();
```

Step 8: Configure Application

```
# .env file
```

```
DATABASE_URL=postgresql://neuronest_admin:your_password@your-host:5432/neuronest
```

Step 9: Verify Installation

```
SELECT table_name FROM information_schema.tables  
WHERE table_schema = 'public' ORDER BY table_name;  
-- Expected: 7 tables listed
```

Document generated for NeuroNest v2.0.0 — Academic Year 2025–2026